

NYKAA AUTOMATION

Done by Badrinadh CH

Table of Contents

1. Project Overview
2. Folder Structure
3. Test Configuration
4. Test Data
5. Helper Functions
6. Positive Test Cases
7. Negative Test Cases
8. Logging & Screenshots
9. How to Run
10. Expected Results Summary

1. Project Overview

This document describes the Selenium-based automated test suite for the Nykaa Baby Care product listing page. The tests validate filtering, sorting, and end-to-end purchase flow behaviours on the live Nykaa website.

Objectives

- Verify that brand filters correctly narrow down the product listing.
- Verify that discount filters correctly narrow down the product listing.
- Validate an end-to-end flow from filter selection through to the product detail page.
- Confirm that invalid filter values are gracefully handled or absent.
- Capture screenshots and structured logs for every test step.

Technology Stack

Component	Technology / Library
Language	Python 3.x
Test Runner	Pytest
Browser Automation	Selenium WebDriver
Browser	Google Chrome (ChromeDriver)
Reporting	Allure Framework
Logging	Python logging module (file + console)
Screenshots	Selenium save_screenshot + Allure attachment

2. Folder Structure

The project follows a clean separation of concerns. Below are the two structures referenced during development:

Pytest Project (nykaa_capstone1)

Folder / File	Purpose
nykaa_mom_baby_selenium/	Root package for all test artefacts
chromedriver-win64/	ChromeDriver binary for Windows 64-bit
config/	Configuration files (URLs, timeouts, credentials)
logs/	Execution log files written by the logging module
pages/	Page Object Model classes (one file per page)
reports/	Allure and HTML report output directory
screenshots/	Timestamped PNG screenshots from each test step
test_data/	External test data files (JSON / CSV / Excel)
tests/	All pytest test modules
utils/	Shared utility functions and helpers
conftest.py	Pytest fixtures – WebDriver setup & teardown
chromedriver.exe	ChromeDriver executable (root level copy)
requirements.txt	Python dependency list

BDD Project (BDD_Framework)

Folder / File	Purpose
BDD/config/	Behave configuration and init.py
BDD/features/e2e/	End-to-end .feature files
BDD/features/negative/	Negative scenario .feature files
BDD/features/positive/	Positive scenario .feature files
BDD/features/steps/	Step definition Python files
BDD/features/environment.py	Behave hooks (before/after scenario)
BDD/logs/	Behave execution logs
BDD/pages/	Page Object Model shared with Behave
BDD/reports/	Behave HTML/JSON reports
BDD/testdata/	Test data for BDD scenarios

Folder / File	Purpose
BDD/utills/	Shared utilities
behave.ini	Behave runner configuration
run_tests.py	Entry point to run all Behave suites
requirements.txt	BDD project dependencies

3. Test Configuration

All runtime constants are defined at the top of the test module and can be overridden via environment variables or a central config file.

Constant	Value	Description
BABY_CARE_URL	https://www.nykaa.com/baby-care/c/14798	Target page URL
BRAND_FILTER_DATA	Mamaearth, Himalaya, Cetaphil	Parametrised brand values
DISCOUNT_FILTER_DATA	10% And Above, 20% And Above	Parametrised discount values
INVALID_BRAND_DATA	XYZ123InvalidBrand, NoSuchBrandABC	Brands expected to be absent
Implicit wait	time.sleep(4) on page load	Navigation settle time
Explicit wait	WebDriverWait(driver, 10)	Element wait timeout

4. Test Data

Test data is declared as `pytest.param` lists at module level. This enables parametrised execution where each value becomes a separate test run with a human-readable ID in the Allure report.

Brand Filter Data

Param ID	Brand Name	Expected Behaviour
brand_Mamaearth	Mamaearth	Products reduced to Mamaearth only
brand_Himalaya	Himalaya	Products reduced to Himalaya only
brand_Cetaphil	Cetaphil	Products reduced to Cetaphil only

Discount Filter Data

Param ID	Discount Value	Expected Behaviour
discount_10	10% And Above	Products with at least 10 % discount shown
discount_20	20% And Above	Products with at least 20 % discount shown

Invalid Brand Data

Param ID	Invalid Brand	Expected Behaviour
invalid_XYZ123	XYZ123InvalidBrand	Brand absent from filter panel
invalid_ABC	NoSuchBrandABC	Brand absent from filter panel

5. Helper Functions

All reusable browser interactions are extracted into standalone helper functions. This prevents code duplication and simplifies maintenance.

open_baby_care(driver)

Navigates to the Baby Care category URL, waits 4 seconds for the page to settle, then asserts that the URL contains 'baby-care' and the page title contains 'Nykaa'.

get_product_count(driver)

Locates all anchor tags matching the XPath `//a[contains(@href, '/p/')]` and returns the count. Used before and after filter actions to verify product count changes.

expand_filter(driver, filter_name)

Uses `WebDriverWait(10s)` to find the filter heading by normalised text, scrolls it into view, and clicks via JavaScript. Returns True on success, False on timeout.

click_filter_option(driver, option)

Uses `WebDriverWait(10s)` to find a label/span/div containing the option text, scrolls into view, clicks via JavaScript, and waits 4 seconds for results to refresh. Returns True on success, False on exception.

apply_sort(driver, sort_text)

Clicks the Sort By control to open the dropdown, then clicks the matching sort option. Waits 4 seconds after selection. Returns True on success.

safe_screenshot(driver, name)

Saves a timestamped PNG to the screenshots/directory and attaches it to the current Allure test report node. Silently logs a warning if the screenshot fails.

6. Positive Test Cases

TC_01 – Brand Filter (Parametrised)

Class: TestPositiveCases **Method:** test_TC_01_brand_filter

Field	Detail
Runs	3 (one per brand: Mamaearth, Himalaya, Cetaphil)
Priority	High
Precondition	Baby Care page loads successfully

Steps

- Step 1: Open Baby Care page and record initial product count.
- Step 2: Capture screenshot of the opened page.
- Step 3: Assert initial product count > 0 and URL is HTTPS.
- Step 4: Scroll down 300 px and expand the Brand filter section.
- Step 5: Capture screenshot of expanded filter.
- Step 6: Click the parametrised brand checkbox.
- Step 7: Capture screenshot of selected brand.
- Step 8: Record product count after filtering.
- Step 9: Capture screenshot of filtered results.

Assertions

- count_before >0
- URL starts with 'https'
- Brand filter section expanded (expand_filter returns True)
- Brand option clicked (click_filter_option returns True)
- count_after >0
- count_after <= count_before
- document.readyState == 'complete'

TC_02 – Discount Filter (Parametrised)

Class: TestPositiveCases **Method:** test_TC_02_discount_filter

Field	Detail
Runs	2 (10% And Above, 20% And Above)
Priority	High
Precondition	Baby Care page loads successfully

Steps

- Step 1: Open Baby Care page, record initial product count.

- Step 2: Capture screenshot.
- Step 3: Scroll down 300 px and expand the Discount filter section.
- Step 4: Click the parametrised discount option (retries lowercase if first click fails).
- Step 5: Capture screenshot after selection.
- Step 6: Record product count after filtering.
- Step 7: Capture screenshot of results.

Assertions

- count_before >0
- Discount filter section expanded
- Discount option clicked (with case-insensitive fallback)
- count_after >0
- count_after <= count_before

TC_03 – End-to-End Flow

Class: TestPositiveCases **Method:** test_TC_03_three_filters_proceed_to_payment

Field	Detail
Runs	1 (single combined scenario)
Priority	Critical
Precondition	Baby Care page loads; Mamaearth products exist with 10% discount

Steps

- Step 1: Open Baby Care page and capture screenshot.
- Step 2: Expand Brand filter and select 'Mamaearth'.
- Step 3: Capture screenshot after brand selection.
- Step 4: Expand Discount filter and select '10% And Above' (retries lowercase).
- Step 5: Capture screenshot after discount selection.
- Step 6: Apply Sort by 'Popularity'.
- Step 7: Capture screenshot after sort applied.
- Step 8: Get total product count.
- Step 9: Collect all product anchor links and validate the first one.
- Step 10: Navigate to the first product detail page.
- Step 11: Capture screenshot of product page.
- Step 12: Read page body text.

Assertions

- Brand filter expanded
- Mamaearth selected
- Discount filter expanded

- Discount option selected
- Sort applied (Popularity)
- Product count >0
- At least one product link found
- First product URL is not None and contains '/p/'
- Current URL after navigation contains '/p/'
- Page body contains 'ADD TO BAG' or 'ADD TO CART'

7. Negative Test Cases

TC_04 – Invalid Brand Validation (Parametrised)

Class: TestNegativeCases **Method:** test_TC_04_invalid_brand_search

Field	Detail
Runs	2 (XYZ123InvalidBrand, NoSuchBrandABC)
Priority	Medium
Purpose	Confirm that nonsense brand names do not appear on the filter panel

Steps

- Step 1: Open Baby Care page.
- Step 2: Scroll down 300 px and expand the Brand filter section.
- Step 3: Read the full text content of the page body.
- Step 4: Capture screenshot.

Assertions

- Brand filter section expanded
- Invalid brand name (lowercase) NOT present in body text

TC_05 – Invalid Discount Option Rejection

Class: TestNegativeCases **Method:** test_TC_05_invalid_filter_rejection

Field	Detail
Runs	1
Priority	Medium
Purpose	Confirm that '150% And Above' does not exist as a discount option

Steps

- Step 1: Open Baby Care page.
- Step 2: Scroll down 300 px and expand the Discount filter section.
- Step 3: Attempt to click the '150% And Above' option.
- Step 4: Capture screenshot.

Assertions

- Discount filter section expanded
- click_filter_option returns False or None (option does not exist)

8. Logging & Screenshots

Every test execution produces two parallel outputs: a structured log file and timestamped screenshots. Both are also attached to the Allure report.

Log File

Setting	Value
File path	logs/execution.log
Mode	Append (mode='a')
Encoding	UTF-8
Level	INFO
Format	YYYY-MM-DD HH:MM:SS [LEVEL] Message
Handlers	FileHandler + StreamHandler (console)

Example log lines:

```
2024-09-12 10:23:41 [INFO] BabyCarepageopenedsuccessfully
2024-09-12 10:23:45 [INFO] Filter section expanded: Brand
2024-09-12 10:23:49 [INFO] Filter option selected: Mamaearth
2024-09-12 10:23:53 [INFO] Product count on page: 18
2024-09-12 10:23:53 [INFO] TC_01 PASSED - Mamaearth
```

Screenshots

The `safe_screenshot()` function saves a PNG with a timestamp suffix and attaches it directly to the Allure report node for the running test.

Screenshot Name Pattern	Taken At
TC_01_{brand}_page_opened	After page navigation
TC_01_{brand}_filter_expanded	After Brand filter expanded
TC_01_{brand}_selected	After brand option clicked
TC_01_{brand}_result	After filtered results displayed
TC_02_{discount}_page_opened	After page navigation
TC_02_{discount}_selected	After discount option clicked
TC_02_{discount}_result	After filtered results displayed
TC_03_page_opened	After page navigation
TC_03_brand_selected	After Mamaearth selected
TC_03_discount_selected	After discount selected
TC_03_sort_applied	After sort applied

Screenshot Name Pattern	Taken At
TC_03_product_page	After navigating to product detail
TC_04_{invalid_brand}	After Brand filter expanded
TC_05_invalid_discount	After discount click attempt

9. How to Run

Prerequisites

- Python 3.9 or higher installed.
- Google Chrome installed (version matching ChromeDriver).
- ChromeDriver placed in PATH or in the project root.
- All dependencies installed: `pip install -r requirements.txt`
- Allure command-line tool installed for report generation.

Install Dependencies

```
pip install -r requirements.txt
```

Run All Tests

```
pytest tests/ -v --alluredir=reports/allure-results
```

Run Specific Test Class

```
# Positive cases only
pytest tests/ -k TestPositiveCases -v

# Negative cases only
pytest tests/ -k TestNegativeCases -v
```

Run a Single Test Case

```
pytest tests/ -k test_TC_01_brand_filter -v
pytest tests/ -k test_TC_03_three_filters_proceed_to_payment -v
```

Generate Allure Report

```
allureserverreports/allure-results #
or
allure generate reports/allure-results -o reports/allure-html --clean
```

10. Expected Results Summary

The table below summarises each test case, its type, parametrisation, and expected pass/fail condition.

TC ID	Test Name	Type	Runs	Pass Condition
TC_01	Brand Filter	Positive	3	Productsfilteredperbrand, count reduced
TC_02	Discount Filter	Positive	2	Productsfilteredbydiscount, count reduced
TC_03	End-to-End Flow	Positive	1	ProductdetailpageshowsAdd to Bag
TC_04	Invalid Brand Search	Negative	2	Invalidbrandabsentfromfilter panel
TC_05	Invalid Discount Option	Negative	1	150%optionnotclickable/ absent

Overall Test Count

Category	Count
Positive Test Runs	6 (3 brand + 2 discount + 1 e2e)
Negative Test Runs	3 (2 invalid brand + 1 invalid discount)
Total Test Runs	9